

Module-02

☐ Simple Sieve

☐ Segmented Sieve

☐ Euler's phi Algorithm

☐ Remainder Theorem

☐ Strobogrammatic Number

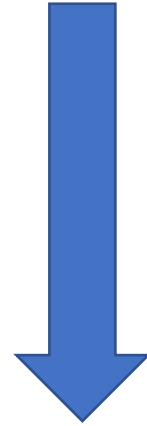
☐ Binary Palindrome

☐ Booth's Algorithm

- Write a java program for finding a number is prime or not.

1. Time complexity should be $O(n)$.

we will try to reduce it to



2. Time Complexity should be $O(\sqrt{n})$

```
1.  import java.util.Scanner;
2.  public class Main {
3.  public static void main(String[] args) {
4.      Scanner sc=new Scanner(System.in);
5.      int n=sc.nextInt();
6.      for(int i=2;i<=n;i++)
7.      {
8.          if(n%i==0)
9.          {
10.             System.out.print("not a prime");
11.             return;
12.          }
13.      }
14.      System.out.print("prime");
15.      return;
16.  }
17. }
```

Time Complexity – $O(n)$

```
1.  import java.util.Scanner;
2.  public class Main
3.  {
4.      public static void main(String[] args) {
5.          Scanner sc=new Scanner(System.in);
6.          int n=sc.nextInt();
7.          for(int i=2;i*i<=n;i++)
8.          {
9.              if(n%i==0)
10.             {
11.                 System.out.print("not a prime");
12.                 return;
13.             }
14.         }
15.         System.out.print("prime");
16.         return;
17.     }
18. }
```

Time Complexity – $O(\sqrt{n})$

- **Sieve of Eratosthenes** is a method to find the prime numbers and composite numbers among a group of numbers.
- Prime Numbers?
- Composite Numbers?

- What are all prime numbers less than 20. (Use the Sieve of Eratosthenes method).

❖ Now, exclude 1 since it is not a prime nor a composite number.

❖ In the next step, encircle 2 and cross the multiples of 2 (4, 6, 8, 10, 12, 14, 16, 18, 20)

❖ Encircle 3 and cross the left multiples of 3 (9, 15)

❖ Encircle 5 and cross the left multiples of 5 (only 10 is left)

❖ Now, encircle the rest of the numbers 7, 11, 13, 17 and 19.

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20

Steps of Sieve's Algorithm

- Step-1 Take a Boolean array of size like up to what range you want to know the prime numbers.
- Step-2 Except 0 & 1 index of the array mark all the remaining elements as "true".
- Step-3 Start from 2 cross out all the multiples of 2 and then for 3 and so on..
- Step-4 Take a loop and print wherever array contains true value.

Code

```
1.  public static void sieve(int n)
2.  {
3.      boolean p[]=new boolean[n+1];
4.      for(int i=2;i<=n;i++)
5.          p[i]=true;
6.      for(int i=2;i*i<=n;i++) {
7.          for(int j=i*i;j<=n;j=j+i) {
8.              if(p[j]==true)
9.                  p[j]=false;
10.         }
11.     }
12.     for(int i=2;i<=n;i++) {
13.         if(p[i])
14.             System.out.print(i+" ");
15.     }
16. }
```

Time Complexity

- For each prime p we are marking $2p, 3p, 4p, \dots, kp$.
- $n/2 + n/3 + n/4 + \dots$
- $n(1/2 + 1/3 + 1/4 + \dots)$
- $n(\log(\log n))$
- Hence **$O(n \log(\log n))$**

Euclid's Algorithm

- GCD can be found in two ways:
 - i. Traditional Method
 - ii. Euclid's Algorithm

Traditional Method

Example : gcd (12,33) → ?

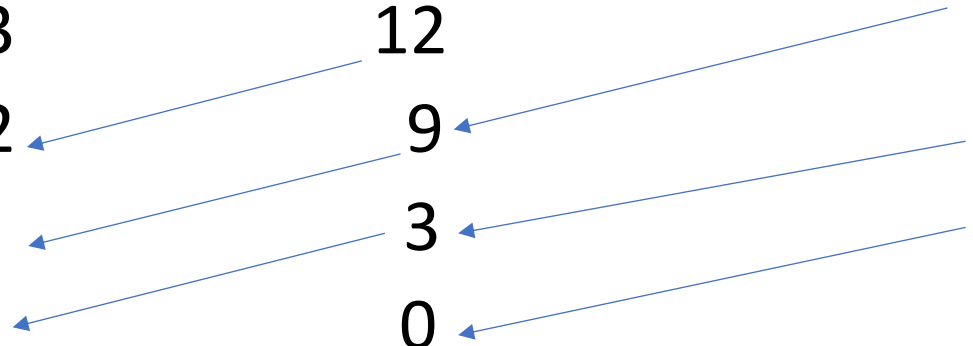
Given Numbers	12	33
Divisors -----	1,2,3,4,6,12	1,3,11,33
Common Divisors -----	1,3	
Greatest Common Divisors-----	3	

Euclid Algorithm:

Example $\text{gcd}(12,33)$ ----->?

$\text{Max}(12,33)$ will be given to a

quotient	a	b	remainder
2	33	12	9
1	12	9	3
3	9	3	0
	3	0	



Stop the process as b is '0' and whatever the 'a' contains that is the gcd of the two numbers.

$$\text{gcd}(12,33)=3$$

```
1.  import java.util.Scanner;
2.  class Main{
3.      public static int gcd(int a,int b)
4.      {
5.          if(b==0)
6.              return a;
7.          return gcd(b,a%b);
8.      }
9.      public static void main (String[] args) {
10.          Scanner sc=new Scanner(System.in);
11.          int n=sc.nextInt();
12.          int m=sc.nextInt();
13.          System.out.print(gcd(n,m));
14.      }
15. }
```

Euler's Phi Algorithm

Euler's Phi Function

- Denoted by $\phi(n)$
- Euler's phi (or totient) function of a positive integer n is the number of integers in $\{1, 2, 3, \dots, n\}$ which are relatively prime to n .

Relatively Prime:

Two numbers are said to be **relatively prime** if their greatest common factor (GCD or HCF) is 1 .

- Example-1

$$\phi(n) \rightarrow \phi(5)$$

here $n = 5$

So for the numbers that are less than 5 i.e. 1,2,3 & 4 for these numbers we have to check whether gcd is 1 or not and finally we have to count how many pairs of n gcd is 1 and we have to say the count that is called as eulers phi function.

$$\text{Gcd}(1,5) \rightarrow 1$$

$$\text{Gcd}(2,5) \rightarrow 1$$

$$\text{Gcd}(3,5) \rightarrow 1$$

$$\text{Gcd}(4,5) \rightarrow 1$$

Therefore $\phi(5)$ is 4.

} 4

- Example-2

$$\phi(8) \longrightarrow \text{?????}$$

We have to check number with 1,2,3,4,5,6,7.

$$\text{Gcd}(1,8)=1$$

$$\text{Gcd}(2,8)=2$$

$$\text{Gcd}(3,8)=1$$

$$\text{Gcd}(4,8)=4$$

$$\text{Gcd}(5,8)=1$$

$$\text{Gcd}(6,8)=2$$

$$\text{Gcd}(7,8)=1$$

we have to count only pairs which give gcd as 1 so
 $\phi(8) \rightarrow 4$.

- So if the input number is large it is difficult to find all the relative prime pairs that is reason Euler introduced 3 formulas through which we can easily tell the relatively prime pairs count.

- If the given number 'n' is prime then $\phi(n) \Rightarrow n - 1$
- If the given number is not a prime number but we can represent 'n' as product of primes then $\phi(n) = (p-1)(q-1)$ here $p \neq q$.
- If the above two failed then find the prime factors and substitute in the below formula:

$$\phi(n) = n * \left(1 - \frac{1}{p_1}\right) * \left(1 - \frac{1}{p_2}\right) \dots \dots \dots \left(1 - \frac{1}{p_k}\right)$$

Examples:

1. num=7

As num is prime we can substitute in $\phi(\text{num})=\text{num}-1$ formula

So $7-1=6$ relatively primes are there for 7.

2. num=21

Here num is not a prime number but this can be represented in the product of primes form i.e. $7*3$

So $(7-1)(3-1)=6(2)=12$ relatively primes are there for 21.

3. num=1000

Prime factors of 1000 are $2^3 * 5^3$

So after removing powers we will remain with 2 and 5 and now apply 3rd formula we will get 400 as the result.

Practice Questions

- i. 5
- ii. 31
- iii. 35
- iv. 7000
- v. 369
- vi. 372

A handwritten diagram showing the prime factorization of 1000. It consists of a vertical stack of division steps. At the top, '2' is written to the left of a horizontal line, with '1000' above it. Below the line is '500'. This is repeated: '2' to the left of a line, '500' above, '250' below. Then '5' to the left of a line, '250' above, '50' below. Then '5' to the left of a line, '50' above, '10' below. Finally, '2' to the left of a line, '10' above, and '5' below. The final '5' is not under a line.

```
1. import java.util.*;
2. public class Eulers {
3.     public static void main(String args[]) {
4.         Scanner sc = new Scanner(System.in);
5.         int num = sc.nextInt();
6.         int a = 1, count = 0;
7.         for (a = 1; a < num; a++) {
8.             if (gcd(a, num) == 1)
9.                 count++; }
10.        System.out.print(count);
11.    }
12.    public static int gcd(int a, int b) {
13.        if (a == 0)
14.            return b;
15.        return gcd(b % a, a); } } }
```

Chinese Remainder Theorem

Chinese Remainder Theorem

- It is used to solve a set of different congruent equations with one variable but different modulo which are relatively prime.
- $x \equiv a_1(\text{mod } m_1),$
 $x \equiv a_2(\text{mod } m_2),$
▪
▪
▪
 $x \equiv a_n(\text{mod } m_n).$
- Where one variable i.e common variable is 'x' which we have to find with the help of different modulo $m_1, m_2, m_3, \dots, m_n$ and $a_1, a_2, \dots, a_n$.
- Check Whether $(m_1, m_2, \dots, m_n)$ are relatively prime or not.

- Substitute the values after finding M_1 , M_2 , M_3 and M_1^{-1} , M_2^{-1} , M_3^{-1} .

$$x = (a_1 * M_1 * M_1^{-1} + a_2 * M_2 * M_2^{-1} + a_3 * M_3 * M_3^{-1} + \dots + a_n * M_n * M_n^{-1}) \bmod M$$

Steps :

- Step 1: Calculate the value of **M** where **M = m1*m2*m3**.
- Step 2: Find **M₁**, **M₂**, **M₃** where **M₁ = M/m1**, **M₂ = M/m2**, **M₃ = M/m3**.
- Step 3: Find **M₁⁻¹**, **M₂⁻¹**, **M₃⁻¹** which are the modular multiplicative inverses of **M₁**, **M₂**, **M₃** respectively using the congruence relations give below.

$$\mathbf{M_1 * M_1^{-1} \equiv 1(mod\ m1)}$$

$$\mathbf{M_2 * M_2^{-1} \equiv 1(mod\ m2)}$$

$$\mathbf{M_3 * M_3^{-1} \equiv 1(mod\ m3)}$$

- Step 4:

$$\mathbf{x=(a_1*M_1* M_1^{-1} + a_2*M_2* M_2^{-1} + a_3*M_3* M_3^{-1} +.....+ a_n*M_n* M_n^{-1})\ mod\ M}$$

Example - 01

- $x \equiv 2 \pmod{3}$
 $x \equiv 3 \pmod{5}$
 $x \equiv 2 \pmod{7}$

Solution:

Step→01

$$M = m_1 * m_2 * m_3 \Rightarrow 3 * 5 * 7 \rightarrow 105$$

Step→02

$$M_1 = M/m_1 \rightarrow 105/3 \rightarrow 35$$

$$M_2 = M/m_2 \rightarrow 105/5 \rightarrow 21$$

$$M_3 = M/m_3 \rightarrow 105/7 \rightarrow 15$$

• Step → 03

$$M_1 * M_1^{-1} = 1 \bmod m_1$$

$$M_2 * M_2^{-1} = 1 \bmod m_2$$

$$M_3 * M_3^{-1} = 1 \bmod m_3$$

$$35 * (1) \neq 1 \bmod 3 \quad \times$$

$$35 * (2) = 1 \bmod 3 \quad \checkmark$$

$$\text{So } M_1^{-1} = 2$$

$$21 * (1) = 1 \bmod 5 \quad \checkmark$$

$$\text{So } M_2^{-1} = 1$$

$$15 * (1) = 1 \bmod 7 \quad \checkmark$$

$$\text{So } M_3^{-1} = 1$$

• Step→04

$$x=(a_1*M_1* M_1^{-1} + a_2*M_2* M_2^{-1} + a_3*M_3* M_3^{-1} +.....+ a_n*M_n* M_n^{-1})$$

$$x=(2*35*2)+(3*21*1)+(2*15*1)$$

$$x=140+63+30$$

$$x=233$$

Step→05

$$X=x \bmod M$$

$$X=233 \bmod 105$$

$$X=23$$

Verifying.....

- $x \equiv 2 \pmod{3}$
 $x \equiv 3 \pmod{5}$
 $x \equiv 2 \pmod{7}$

Now as now value is 23 substitute that and check whether it is getting the remainder as given in the question or not.

$$\begin{array}{r} 3 \overline{)23} \text{ (7} \\ \underline{21} \\ 2 \end{array} \checkmark$$

- $23 \equiv 2 \pmod{3}$

$$\begin{array}{r} 5 \overline{)23} \text{ (4} \\ \underline{20} \\ 3 \end{array} \checkmark$$

$$23 \equiv 3 \pmod{5}$$

$$\begin{array}{r} 7 \overline{)23} \text{ (3} \\ \underline{21} \\ 2 \end{array} \checkmark$$

$$23 \equiv 2 \pmod{7}$$

Practice questions:

1. $x \equiv 8 \pmod{9}$
 $x \equiv 3 \pmod{20}$

2. $x \equiv 5 \pmod{3}$
 $x \equiv 2 \pmod{5}$
 $x \equiv 1 \pmod{11}$

3. $x \equiv 1(\text{mod}3)$
 $x \equiv 4(\text{mod}5)$
 $x \equiv 6(\text{mod}7)$

4. $x \equiv 2(\text{mod}3)$
 $x \equiv 3(\text{mod}8).$

Binary Palindrome

Binary Palindrome

- Problem Statement:

Take a number and convert it into binary form then check whether it is palindrome or not.

Binary Palindrome Examples:

0, 1, 11, 101, 111, 1001, 1111, 10001, 10101, 11011, 11111, 100001, ...

```
1.  public static boolean isbp(int n)
2.  {
3.      if(n==0) return true;
4.      int len=(int) (Math.log(n)/Math.log(2))+ 1;
5.      int left=len-1, right=0;
6.      while(left>right)
7.      { int l=(n>>left)&1;
8.          int r=(n>>right)&1;
9.          if(l!=r)
10.         {
11.             return false;
12.         }
13.         left--;
14.         right++;
15.     }
16.     return true;
17. }
```

Strobogrammatic Number

Strobogrammatic Number:

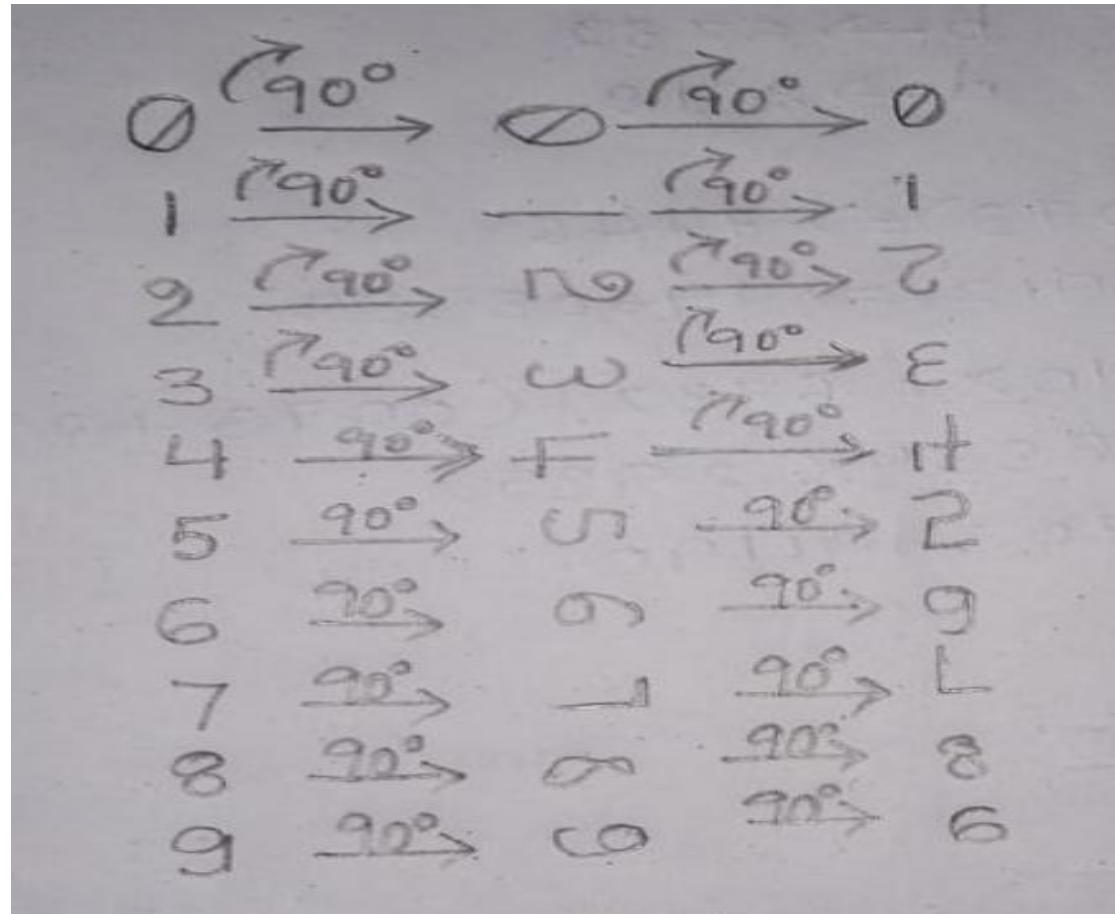
- It is a number that looks same when rotated 180 degrees.

Examples : 69,96,101,88,181,609,916.....

- When rotating digits it may falls under any of the following case:
 - i. Invalid
 - ii. Stays Same
 - iii. Different Digit

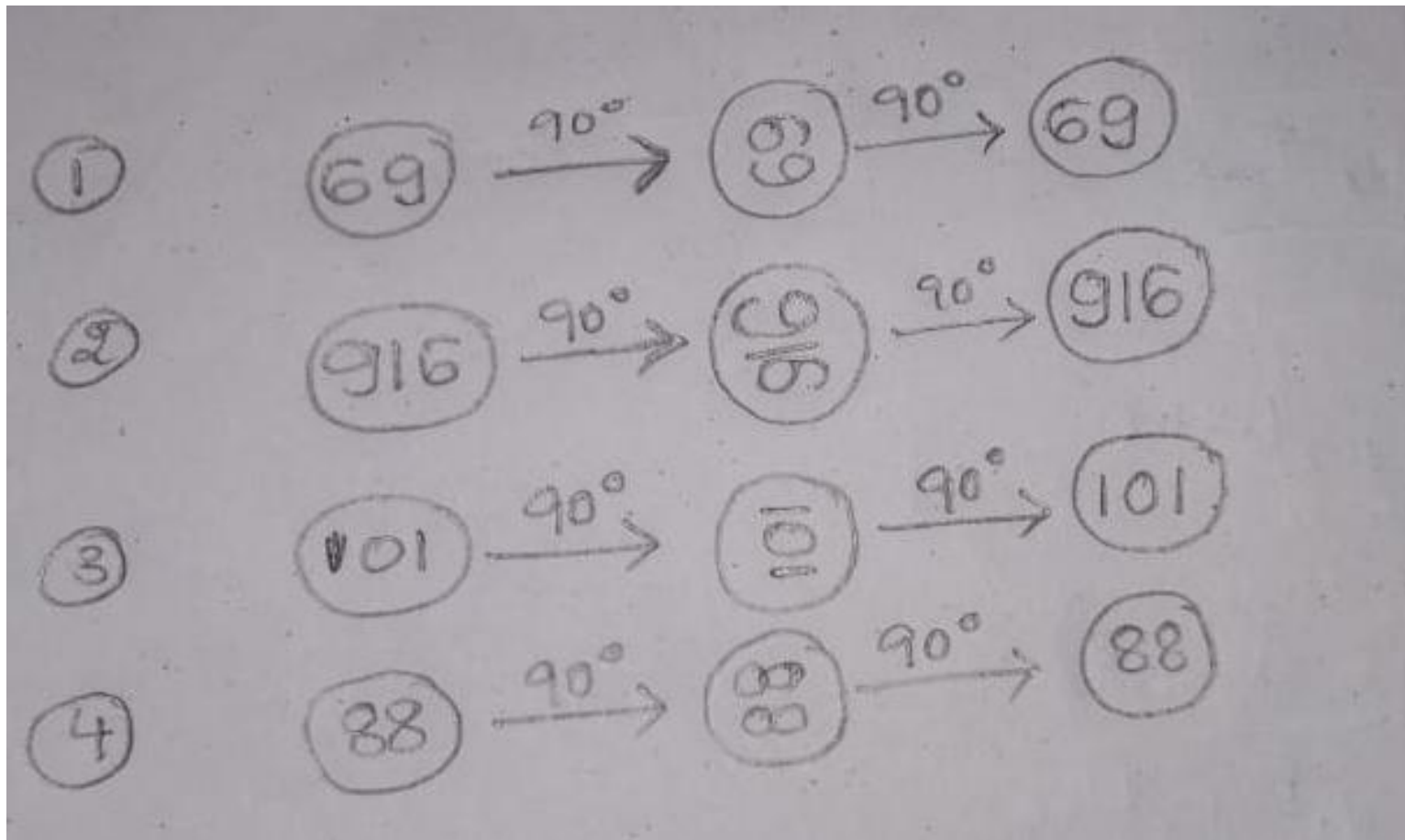
If `num.length()==1`

- Strobogrammatic Number list is:-[1,0,8]



If `num.length() > 1`

- Strobogrammatic Number list is:-[1,0,8,6,9]



- Logic is given below
- If the

left part is right part should contain

$$0 \text{ ----- } > 0$$
$$1 \text{ ----- } > 1$$

8 ----- > 8

$$6 \text{ ----- } > 9$$

9 ----- > 6

```
1. public static boolean issn(String s)
2. {
3.     int l=0,r=s.length()-1;
4.     while(l<=r)
5.     {
6.         int a=s.charAt(l);
7.         int b=s.charAt(r);
8.         if(a=='0' && b=='0' || a=='1' && b=='1' || a=='8' && b=='8' || a=='6' && b=='9' || a=='9' && b=='6')
9.         {
10.             l++;
11.             r--;
12.         }
13.         else
14.             return false;    }
15.     return true;
16. }
```